CS 300 Pseudocode Document

Example Function Signatures

Below is an example of a function signature that you can use as a guide to help address the program requirements using each data structure for the milestones. The pseudocode for finding and printing course information is also given below and depicted in bold to help you get started. The provided pseudocode is for a vector data structure, so you may use this pseudocode in your first milestone as is. The hash table and tree structures are also shown below. But these structures are left for you to do in future milestones.

```
//Vector - Milestone 1

void searchCourse(Vector<Course> courses, String courseNumber) {
      for all courses
            if the course is the same as courseNumber
                  print out the course information
                  for each prerequisite of the course
                        print the prerequisite course information
}
//Hash Table - Milestone 2

void searchCourse(HashTable<Course> courses, String courseNumber) {

}
```

```
//Binary Search Tree - Milestone 3

void searchCourse(Tree<Course> courses, String courseNumber) {
```

Data Structures

```
STRUCT Course
    courseNumber : string
    title : string
    prereqNumbers : vector<string>
END STRUCT

STRUCT HashNode
    key : string              // normalized courseNumber
    value : Course
    next : HashNode pointer     // for chaining collisions
END STRUCT

STRUCT HashTable
    buckets : vector<HashNode pointer>   // fixed size bucket array
    tableSize : int
END STRUCT
```

```
STRUCT BSTNode
    data : Course
    left : BSTNode pointer
    right : BSTNode pointer
END STRUCT

STRUCT BinarySearchTree
    root : BSTNode pointer
END STRUCT
```

Core Helper Functions

```
FUNCTION NormalizeCourseNumber(rawText : string) RETURNS string
    text <- Trim(rawText)
    text <- ToUpper(text)
    RETURN text
END FUNCTION

FUNCTION SplitLineToTokens(line : string) RETURNS vector<string>
    tokens <- Split(line, ",")
    FOR i FROM 0 TO tokens.size - 1
```

```
        tokens[i] <- Trim(tokens[i])
    ENDFOR
    RETURN tokens
END FUNCTION

FUNCTION PrintCourseLine(courseObj : Course)
    Output(courseObj.courseNumber + ", " + courseObj.title)
END FUNCTION
```

Shared Validation Strategy

All three solutions use the same validation approach so that the file is checked before the data structure is used. The program performs a read and parse loop that builds a temporary map of courses so it can detect duplicates and validate prerequisites. After validation succeeds, the program builds the chosen data structure.

```
FUNCTION LoadAndValidateCourses(filePath : string) RETURNS (bool, map<string, Course>,
vector<string>)
    // Returns:
    // success flag
    // courseMap keyed by normalized courseNumber
    // prereqPairs encoded as "CourseNumber|PrereqNumber" for existence validation
```

```
file <- OpenFileForRead(filePath)
 IF file FAILED TO OPEN THEN
    Output("Error: Cannot open file.")
    RETURN (false, EmptyMap(), EmptyVector())
 ENDIF

courseMap : map<string, Course>
 prereqPairs : vector<string>
 lineNumber <- 0

WHILE NOT EndOfFile(file)
    line <- ReadLine(file)
    lineNumber <- lineNumber + 1
    line <- Trim(line)

   IF line IS EMPTY THEN
       CONTINUE
    ENDIF

   tokens <- SplitLineToTokens(line)

   IF tokens.size < 2 THEN
       Output("Format error on line " + ToString(lineNumber))
```

```
    CloseFile(file)
    RETURN (false, EmptyMap(), EmptyVector())
 ENDIF

courseNum <- NormalizeCourseNumber(tokens[0])
 title <- tokens[1]

IF courseNum IS EMPTY OR title IS EMPTY THEN
    Output("Format error on line " + ToString(lineNumber))
    CloseFile(file)
    RETURN (false, EmptyMap(), EmptyVector())
 ENDIF

IF courseMap CONTAINS KEY courseNum THEN
    Output("Duplicate course number on line " + ToString(lineNumber))
    CloseFile(file)
    RETURN (false, EmptyMap(), EmptyVector())
 ENDIF

c : Course
 c.courseNumber <- courseNum
 c.title <- title
 c.prereqNumbers <- EmptyVector()
```

```
    IF tokens.size > 2 THEN
        FOR i FROM 2 TO tokens.size - 1
            prereq <- NormalizeCourseNumber(tokens[i])
            IF prereq IS EMPTY THEN
                Output("Empty prerequisite token on line " + ToString(lineNumber))
                CloseFile(file)
                RETURN (false, EmptyMap(), EmptyVector())
            ENDIF
            c.prereqNumbers.push_back(prereq)
            prereqPairs.push_back(courseNum + "|" + prereq)
        ENDFOR
    ENDIF

    courseMap[courseNum] <- c
  ENDWHILE

CloseFile(file)

// Validate that every prerequisite exists in the file
  FOR pair IN prereqPairs
      parts <- Split(pair, "|")
      courseNum <- parts[0]
      prereqNum <- parts[1]
```

```
    IF NOT courseMap CONTAINS KEY prereqNum THEN
        Output("Prerequisite " + prereqNum + " listed for " + courseNum + " does not exist in file.")
        RETURN (false, EmptyMap(), EmptyVector())
    ENDIF
  ENDFOR

  RETURN (true, courseMap, prereqPairs)
END FUNCTION
```

Vector Solution

Vector Core Functions

```
FUNCTION Vector_Search(courses : vector<Course>, courseNumber : string) RETURNS Course or null
  target <- NormalizeCourseNumber(courseNumber)
  FOR each courseObj IN courses
    IF courseObj.courseNumber = target THEN
      RETURN courseObj
    ENDIF
  ENDFOR
  RETURN null
END FUNCTION
```

```
FUNCTION Vector_SortByCourseNumber(courses : vector<Course>)
    // Sort in place by alphanumeric courseNumber from lowest to highest
    Sort(courses, CompareByCourseNumberAscending)
END FUNCTION

FUNCTION Vector_PrintAllCoursesSorted(courses : vector<Course>)
    Vector_SortByCourseNumber(courses)
    FOR each courseObj IN courses
        PrintCourseLine(courseObj)
    ENDFOR
END FUNCTION

FUNCTION Vector_PrintCourseWithPrereqs(courses : vector<Course>, courseObj : Course)
    PrintCourseLine(courseObj)

   IF courseObj.prereqNumbers.size = 0 THEN
        Output("Prerequisites: None")
        RETURN
    ENDIF

   Output("Prerequisites:")
    FOR i FROM 0 TO courseObj.prereqNumbers.size - 1
        prereqNum <- NormalizeCourseNumber(courseObj.prereqNumbers[i])
```

```
        prereqCourse <- Vector_Search(courses, prereqNum)


       IF prereqCourse IS null THEN

           Output(prereqNum + " (not found)")

        ELSE

           PrintCourseLine(prereqCourse)

        ENDIF

    ENDFOR

END FUNCTION


FUNCTION Vector_SearchCourseAndPrint(courses : vector<Course>, courseNumber : string)

    courseObj <- Vector_Search(courses, courseNumber)

    IF courseObj IS null THEN

        Output("Course not found: " + NormalizeCourseNumber(courseNumber))

        RETURN

    ENDIF

    Vector_PrintCourseWithPrereqs(courses, courseObj)

END FUNCTION
```

Vector Load and Build

```
FUNCTION LoadCoursesIntoVector(filePath : string, courses : vector<Course>) RETURNS bool
```

```
    courses <- EmptyVector()


   (ok, courseMap, prereqPairs) <- LoadAndValidateCourses(filePath)

    IF ok = false THEN

        RETURN false

    ENDIF


   // Build vector from validated map values

    FOR key IN courseMap KEYS

        courses.push_back(courseMap[key])

    ENDFOR


   RETURN true

END FUNCTION
```

Vector Menu Flow

```
FUNCTION Main_Vector()

    courses : vector<Course>

    fileLoaded <- false

    choice <- 0
```

```
WHILE choice != 9
    Output("1 Load Data Structure")
    Output("2 Print Course List")
    Output("3 Print Course")
    Output("9 Exit")
    choice <- ReadInt()

   IF choice = 1 THEN
      Output("Enter file name:")
      filePath <- ReadString()
      fileLoaded <- LoadCoursesIntoVector(filePath, courses)

   ELSE IF choice = 2 THEN
      IF fileLoaded THEN
         Vector_PrintAllCoursesSorted(courses)
      ELSE
         Output("Please load data first.")
      ENDIF

   ELSE IF choice = 3 THEN
      IF fileLoaded THEN
         Output("Enter course number:")
         courseNum <- ReadString()
         Vector_SearchCourseAndPrint(courses, courseNum)
```

```
        ELSE
            Output("Please load data first.")
        ENDIF
    ENDIF
  ENDWHILE
END FUNCTION
```

Hash Table Solution

Hash Table Core Functions

```
FUNCTION Hash_String(key : string, tableSize : int) RETURNS int
  // Simple deterministic string hash for pseudocode purposes
  hashValue <- 0
  FOR i FROM 0 TO key.length - 1
    hashValue <- (hashValue * 31) + ASCII(key[i])
  ENDFOR
  RETURN hashValue MOD tableSize
END FUNCTION

FUNCTION HashTable_Insert(ht : HashTable, courseObj : Course)
  key <- courseObj.courseNumber
```

```
    index <- Hash_String(key, ht.tableSize)


   // Chaining insert at head
    newNode <- New HashNode()
    newNode.key <- key
    newNode.value <- courseObj
    newNode.next <- ht.buckets[index]
    ht.buckets[index] <- newNode
END FUNCTION


FUNCTION HashTable_Search(ht : HashTable, courseNumber : string) RETURNS Course or null
    target <- NormalizeCourseNumber(courseNumber)
    index <- Hash_String(target, ht.tableSize)


   current <- ht.buckets[index]
    WHILE current IS NOT null
        IF current.key = target THEN
            RETURN current.value
        ENDIF
        current <- current.next
    ENDWHILE


   RETURN null
END FUNCTION
```

```
FUNCTION HashTable_GetAllKeys(ht : HashTable) RETURNS vector<string>
    keys <- EmptyVector()
    FOR i FROM 0 TO ht.tableSize - 1
        current <- ht.buckets[i]
        WHILE current IS NOT null
            keys.push_back(current.key)
            current <- current.next
        ENDWHILE
    ENDFOR
    RETURN keys
END FUNCTION


FUNCTION HashTable_PrintAllCoursesSorted(ht : HashTable)
    keys <- HashTable_GetAllKeys(ht)
    Sort(keys, CompareStringsAscending)


    FOR each key IN keys
        courseObj <- HashTable_Search(ht, key)
        IF courseObj IS NOT null THEN
            PrintCourseLine(courseObj)
        ENDIF
    ENDFOR
END FUNCTION
```

```
FUNCTION HashTable_PrintCourseWithPrereqs(ht : HashTable, courseObj : Course)
    PrintCourseLine(courseObj)

   IF courseObj.prereqNumbers.size = 0 THEN
       Output("Prerequisites: None")
       RETURN
    ENDIF

   Output("Prerequisites:")
    FOR i FROM 0 TO courseObj.prereqNumbers.size - 1
       prereqNum <- NormalizeCourseNumber(courseObj.prereqNumbers[i])
       prereqCourse <- HashTable_Search(ht, prereqNum)

      IF prereqCourse IS null THEN
          Output(prereqNum + " (not found)")
       ELSE
          PrintCourseLine(prereqCourse)
       ENDIF
    ENDFOR
END FUNCTION

FUNCTION HashTable_SearchCourseAndPrint(ht : HashTable, courseNumber : string)
    target <- NormalizeCourseNumber(courseNumber)
```

```
    courseObj <- HashTable_Search(ht, target)


   IF courseObj IS null THEN

       Output("Course not found: " + target)

       RETURN

    ENDIF


   HashTable_PrintCourseWithPrereqs(ht, courseObj)

END FUNCTION
```

Hash Table Load and Build

```
FUNCTION LoadCoursesIntoHashTable(filePath : string, ht : HashTable) RETURNS bool

    // Initialize hash table buckets

    ht.tableSize <- 97

    ht.buckets <- VectorOfNullHashNodes(ht.tableSize)


   (ok, courseMap, prereqPairs) <- LoadAndValidateCourses(filePath)

    IF ok = false THEN

       RETURN false

    ENDIF
```

```
    FOR key IN courseMap KEYS
        HashTable_Insert(ht, courseMap[key])
    ENDFOR

    RETURN true
END FUNCTION
```

Hash Table Menu Flow

```
FUNCTION Main_HashTable()
    ht : HashTable
    fileLoaded <- false
    choice <- 0

    WHILE choice != 9
        Output("1 Load Data Structure")
        Output("2 Print Course List")
        Output("3 Print Course")
        Output("9 Exit")
        choice <- ReadInt()

        IF choice = 1 THEN
```

```
        Output("Enter file name:")
        filePath <- ReadString()
        fileLoaded <- LoadCoursesIntoHashTable(filePath, ht)

    ELSE IF choice = 2 THEN
        IF fileLoaded THEN
            HashTable_PrintAllCoursesSorted(ht)
        ELSE
            Output("Please load data first.")
        ENDIF

    ELSE IF choice = 3 THEN
        IF fileLoaded THEN
            Output("Enter course number:")
            courseNum <- ReadString()
            HashTable_SearchCourseAndPrint(ht, courseNum)
        ELSE
            Output("Please load data first.")
        ENDIF
    ENDIF
  ENDWHILE
END FUNCTION
```

Binary Search Tree Solution

BST Core Functions

```
FUNCTION BST_Insert(tree : BinarySearchTree, courseObj : Course)
    IF tree.root IS null THEN
        node <- New BSTNode()
        node.data <- courseObj
        node.left <- null
        node.right <- null
        tree.root <- node
        RETURN
    ENDIF

   current <- tree.root
    WHILE true
        IF courseObj.courseNumber < current.data.courseNumber THEN
            IF current.left IS null THEN
                node <- New BSTNode()
                node.data <- courseObj
                node.left <- null
                node.right <- null
                current.left <- node
```

```
            RETURN
        ENDIF
        current <- current.left

    ELSE IF courseObj.courseNumber > current.data.courseNumber THEN
        IF current.right IS null THEN
            node <- New BSTNode()
            node.data <- courseObj
            node.left <- null
            node.right <- null
            current.right <- node
            RETURN
        ENDIF
        current <- current.right

    ELSE
        // Duplicate key handling, replace existing
        current.data <- courseObj
        RETURN
    ENDIF
  ENDWHILE
END FUNCTION

FUNCTION BST_Search(tree : BinarySearchTree, courseNumber : string) RETURNS Course or null
```

```
    target <- NormalizeCourseNumber(courseNumber)
    current <- tree.root

   WHILE current IS NOT null
      IF target = current.data.courseNumber THEN
         RETURN current.data
      ELSE IF target < current.data.courseNumber THEN
         current <- current.left
      ELSE
         current <- current.right
      ENDIF
    ENDWHILE

   RETURN null
END FUNCTION

FUNCTION PrintCourseWithPrereqs_BST(tree : BinarySearchTree, courseObj : Course)
    PrintCourseLine(courseObj)

   IF courseObj.prereqNumbers.size = 0 THEN
      Output("Prerequisites: None")
      RETURN
    ENDIF
```

```
    Output("Prerequisites:")
    FOR i FROM 0 TO courseObj.prereqNumbers.size - 1
        prereqNum <- NormalizeCourseNumber(courseObj.prereqNumbers[i])
        prereqCourse <- BST_Search(tree, prereqNum)

       IF prereqCourse IS null THEN
           Output(prereqNum + " (not found)")
        ELSE
           PrintCourseLine(prereqCourse)
        ENDIF
    ENDFOR
END FUNCTION

FUNCTION PrintAllCoursesInOrder(node : BSTNode pointer)
    IF node IS null THEN
        RETURN
    ENDIF
    PrintAllCoursesInOrder(node.left)
    PrintCourseLine(node.data)
    PrintAllCoursesInOrder(node.right)
END FUNCTION

FUNCTION BST_SearchCourseAndPrint(tree : BinarySearchTree, courseNumber : string)
    courseObj <- BST_Search(tree, courseNumber)
```

```
    IF courseObj IS null THEN
        Output("Course not found: " + NormalizeCourseNumber(courseNumber))
        RETURN
    ENDIF

    PrintCourseWithPrereqs_BST(tree, courseObj)
END FUNCTION
```

BST Load and Build

```
FUNCTION LoadCoursesIntoBST(filePath : string, tree : BinarySearchTree) RETURNS bool
    (ok, courseMap, prereqPairs) <- LoadAndValidateCourses(filePath)
    IF ok = false THEN
        RETURN false
    ENDIF

    tree.root <- null
    FOR key IN courseMap KEYS
        BST_Insert(tree, courseMap[key])
    ENDFOR
```

```
    RETURN true
END FUNCTION
```

BST Menu Flow

```
FUNCTION Main_BST()
    bst : BinarySearchTree
    bst.root <- null
    fileLoaded <- false
    choice <- 0

    WHILE choice != 9
        Output("1 Load Data Structure")
        Output("2 Print Course List")
        Output("3 Print Course")
        Output("9 Exit")
        choice <- ReadInt()

        IF choice = 1 THEN
            Output("Enter file name:")
            filePath <- ReadString()
            fileLoaded <- LoadCoursesIntoBST(filePath, bst)
```

```
    ELSE IF choice = 2 THEN
        IF fileLoaded THEN
            PrintAllCoursesInOrder(bst.root)
        ELSE
            Output("Please load data first.")
        ENDIF

    ELSE IF choice = 3 THEN
        IF fileLoaded THEN
            Output("Enter course number:")
            courseNum <- ReadString()
            BST_SearchCourseAndPrint(bst, courseNum)
        ELSE
            Output("Please load data first.")
        ENDIF
    ENDIF
  ENDWHILE
END FUNCTION
```

Runtime Analysis for Load and Build Only

The following runtime analysis evaluates the worst case running time for reading the file, parsing each line, creating Course objects, and inserting them into the data structure. Menu logic and option 2 and option 3 printing and searching are excluded per project instructions. Assume there are n courses in the input file. Cost per simple line is 1. Function call cost equals the runtime of that function.

Vector Load and Build

Code	Line Cost	# Times Executes	Total Cost
for all lines (courses)	1	n	n
read and trim line	1	n	n
split into tokens and trim	1	n	n
validate tokens and build Course object	1	n	n
validate prerequisite existence using map	1	n	n
push Course into vector			n
Total Cost			**6n**

Hash Table Load and Build (Worst case

for all lines (courses)	1	n	n
read and trim line	1	n	n
split into tokens and trim	1	n	n

validate tokens and build Course object	1	n	n
validate prerequisite existence using map	1	n	n
insert into BST (worst case)	n	n	n^2
Total Cost			**$5n + n^2$**
Runtime			**$O(n^2)$**

BST Load and Build (Worst case unbalanced

for all lines (courses)	1	n	n
read and trim line	1	n	n
split into tokens and trim	1	n	n
validate tokens and build Course object	1	n	n
validate prerequisite existence using map	1	n	n
insert into BST (worst case)	n	n	n^2
Total Cost			**$5n + n^2$**
Runtime			**$O(n^2)$**

Evaluation of Data Structures

Vector advantages: simplest structure, fast build, easy to sort for option 2. Vector disadvantages: searching for a single course in option 3 is linear time unless additional indexing is added.

Hash table advantages: fast lookup for option 3 in average case, direct key based access by course number. Hash table disadvantages: unordered, so option 2 requires extracting keys and sorting, and worst case collisions can degrade performance.

Binary search tree advantages: naturally supports alphanumeric ordered printing using in order traversal for option 2, and supports efficient search for option 3 in average case. Binary search tree disadvantages: if the tree is not self balancing, insertion order can produce an unbalanced tree and degrade to worst case behavior.

Recommendation

Recommendation: Use the binary search tree for the advising program implementation. The primary justification is that option 2 requires printing courses in alphanumeric order, which the tree supports directly with an in order traversal without an additional sorting step. Option 3 lookup is efficient in typical cases and matches the course number key model. While the worst case runtime can degrade if the tree becomes unbalanced, the design still aligns best with the program requirements and reduces complexity for producing the sorted course list.